July 2026

06_SAST

Presented by Gabriele Pasqualini

OBJECTIVE

In this lab activity Static Code Analysis will be conducted on 2 cases:

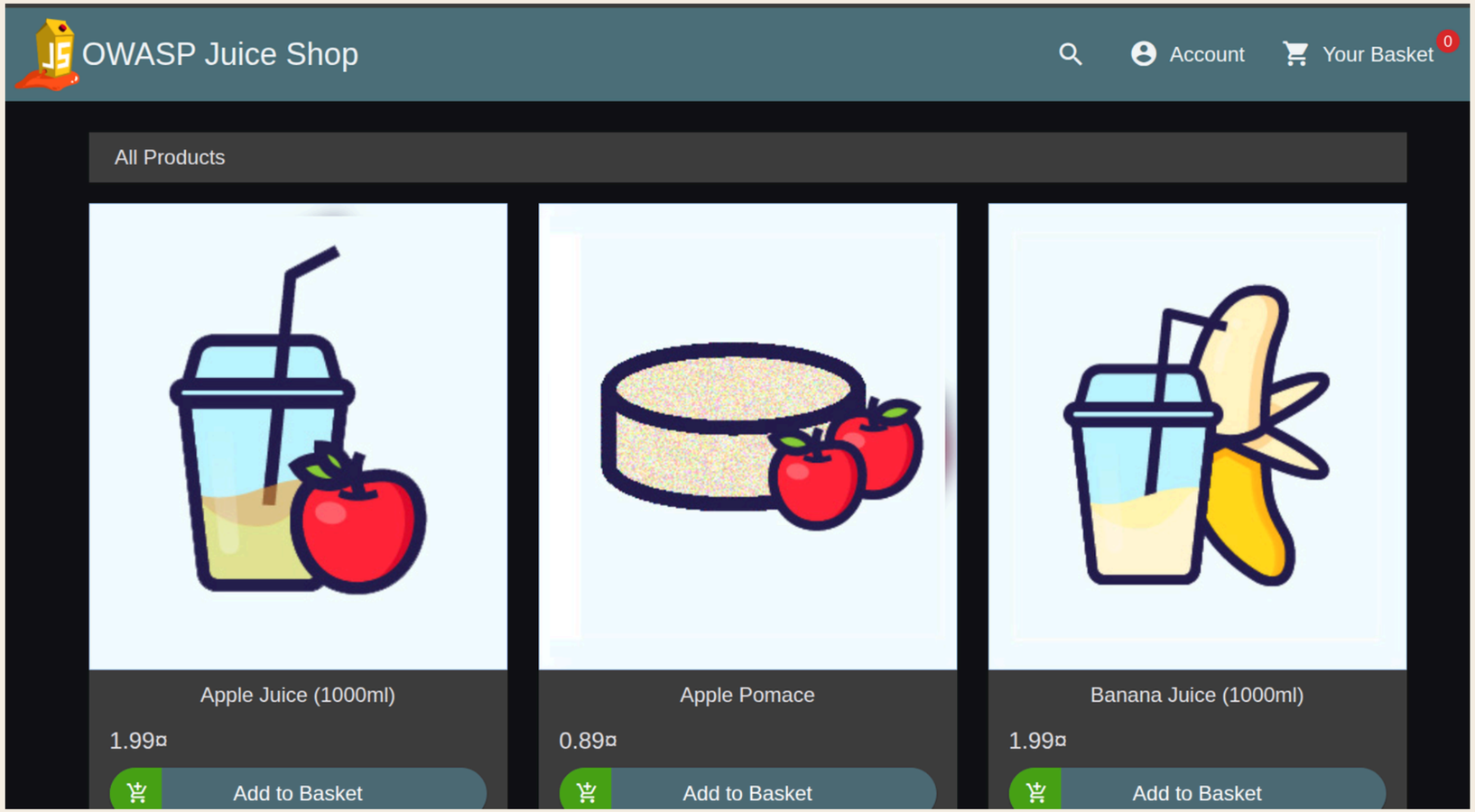
Case 1) one with known vulnerabilities

Case 2) one without known vulnerabilities.

*For each case the code will be analyzed with Sonarcloud for the code and with OWASP's dependency-check tool.

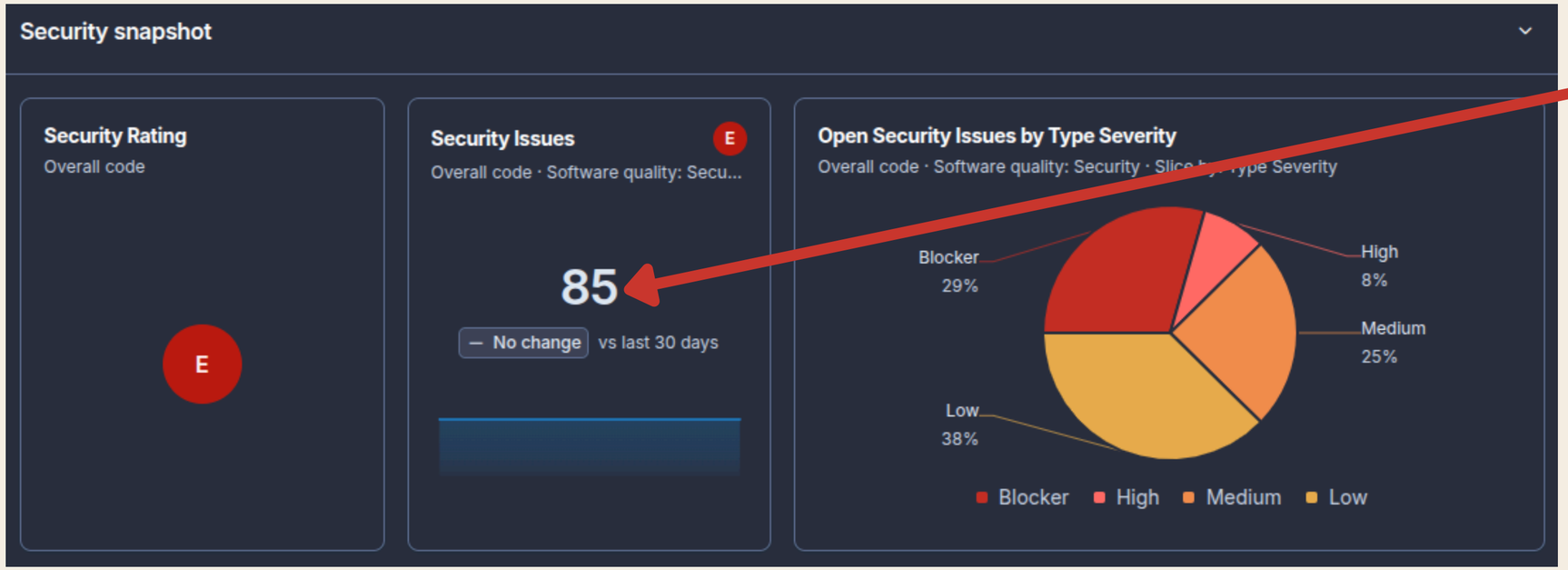
Case 1

As a target application for the security analysis, the intentionally vulnerable repository **OWASP Juice Shop** was selected.



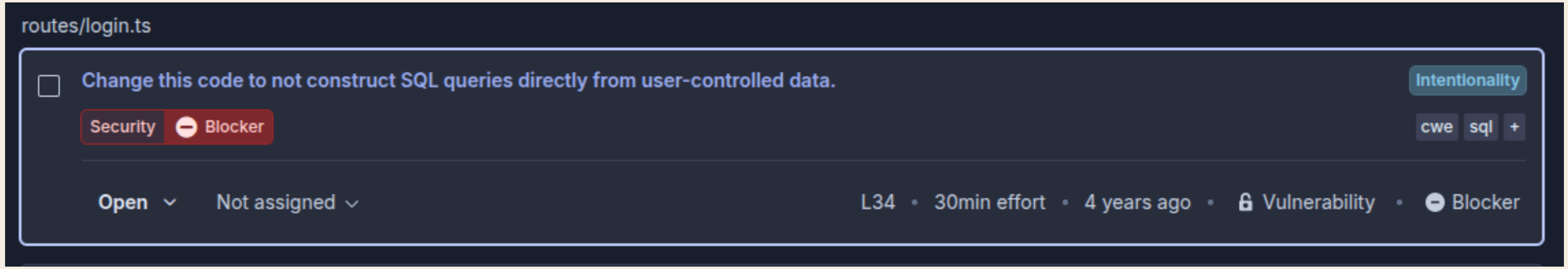
Since it is a quite big repository with many intentional vulnerabilities i decided to focus only on an SQLi challenge

Case 1.1: Sonarcloud Analysis



As expected many Security Issues were found,

Example SQL vulnerability:



Case 1.2: Vulneravle Code Example

```
  return (req: Request, res: Response, next: NextFunction) => {
    verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
    models.sequelize.3 query( 2 `SELECT * FROM Users WHERE email = '${req.body. 1 email || ''}' AND password = '${security.hash(req
.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })
// vuln-code-snippet vuln-line loginAdminChallenge loginBenderChallenge loginJimChallenge
```

Change this code to not construct SQL queries directly from user-controlled data.

Proposed solution template from sonarcloud:

```
async function index(req, res) {
    const knex = req.app.get('knex');

    let loggedInUser = await knex('users')
        .where('user', req.query.user)
        .where('pass', req.query.pass);

    res.send(JSON.stringify(loggedInUser));
    res.end();
}
```

Case 1.3: Dependencies Analysis

- *dependency-check version*: 12.1.8
- *Report Generated On*: Fri, 3 Jul 2026 23:03:22
- *Dependencies Scanned*: 100 (98 unique)
- *Vulnerable Dependencies*: 1
- *Vulnerabilities Found*: 4

HIGH Severity

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
jackson-databind-2.20.0.jar	cpe:2.3:a:fasterxml:jackson-core:2.20.0:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.20.0:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.20.0:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.20.0	HIGH	4	Highest	40

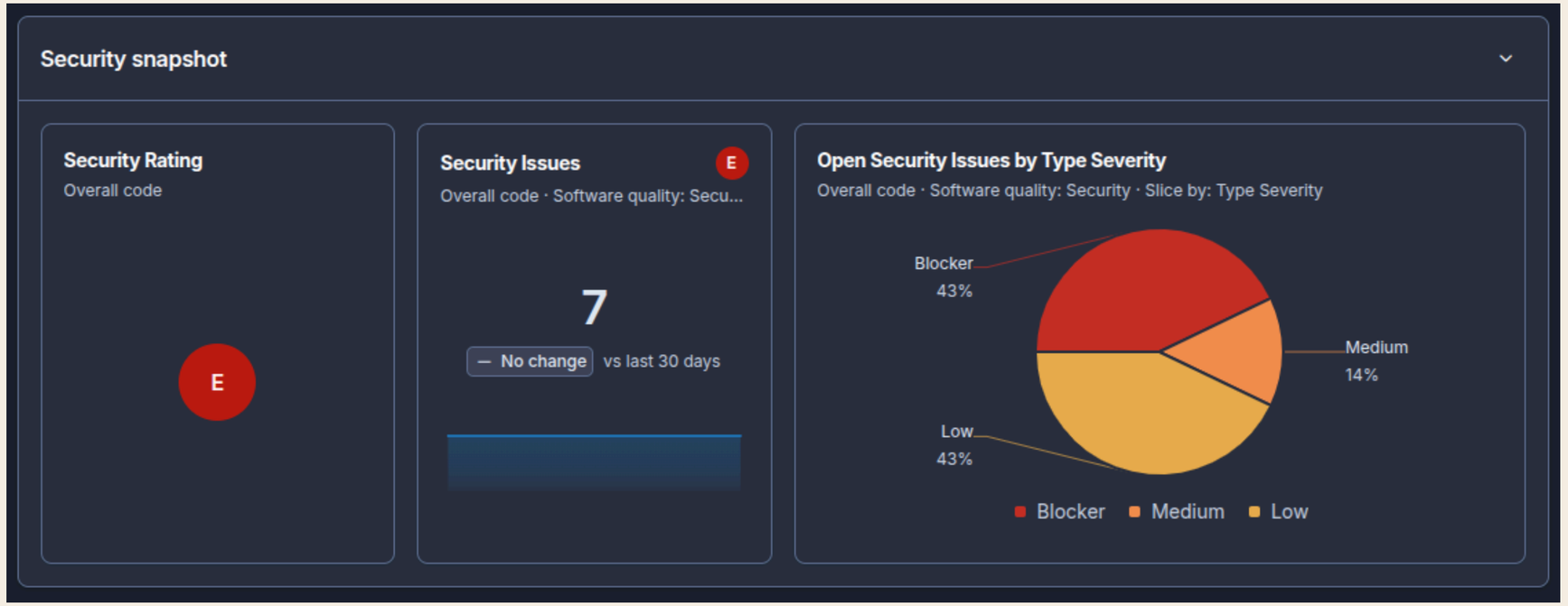
Dependency analysis identified vulnerable versions of the Jackson Databind library. The detected version (2.20.0) is associated with multiple known CVEs classified with HIGH severity.

Case 2

As repository **without** known vulnerabilities my **Web Application Programming Project** was chosen: https://github.com/PasqualiniGabriele/WebApplicationProgramming-Project.

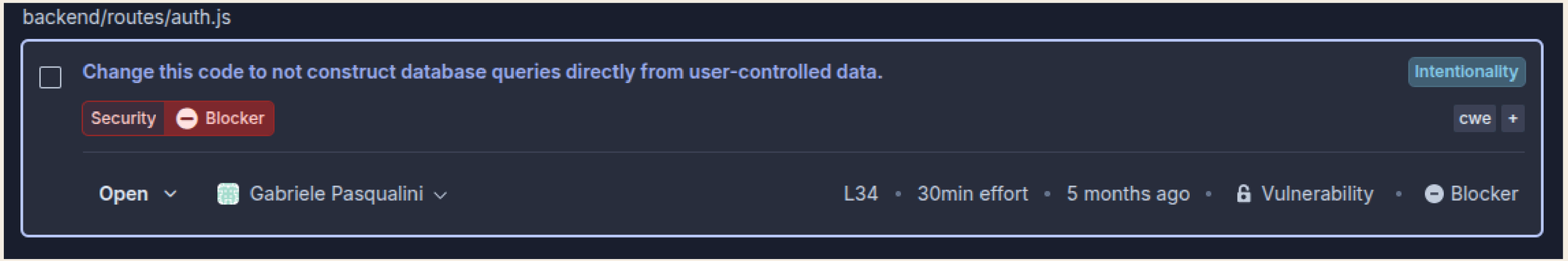
It is a website for sport fields booking and for tournament organization.

Analysis with sonarcloud:



All 3 vulnerabilities found with “**Blocker**” severity are associated with **user input validation**

Case 2.1: Code



Responsible code:

```
// SIGNIN
router.post('/signin', async (req, res) => {
  const { username, password } = req.body;

  const user = await User.findOne({ username });
```

This can lead to a NoSQL Injection vulnerability if an attacker sends a crafted object instead of a normal string, for example using MongoDB operators such as $ne or $gt.

Solution:

The vulnerability was mitigated by validating and sanitizing user input before executing the database query, ensuring that username and password are treated only as strings and rejecting invalid or malformed data to prevent NoSQL injection attacks.

Case 2.2: Dependencies

OWASP dependency-check tool output:

Scan Information (show less):

- *dependency-check version*: 12.1.8
- *Report Generated On*: Fri, 3 Jul 2026 16:31:57
- *Dependencies Scanned*: 203 (117 unique)
- *Vulnerable Dependencies*: 12
- *Vulnerabilities Found*: 25

The analysis of the dependencies highlighted multiple outdated libraries, and for each of those it is sufficient to perform an update.

OBJECTIVE

In this lab activity Static Code Analysis will be conducted on 2 repistories: one with known vulnerabilities and one without known vulnerabilities.